

Módulo 0 — Preparando o computador (material do aluno)

O que vamos fazer hoje

Ainda **não** vamos programar. Hoje a gente deixa o computador **pronto para escrever código** — instala as ferramentas, organiza as pastas e monta o ambiente do projeto. No fim, dá pra começar o Módulo 1 sem parar pra instalar nada.

Você vai entender:

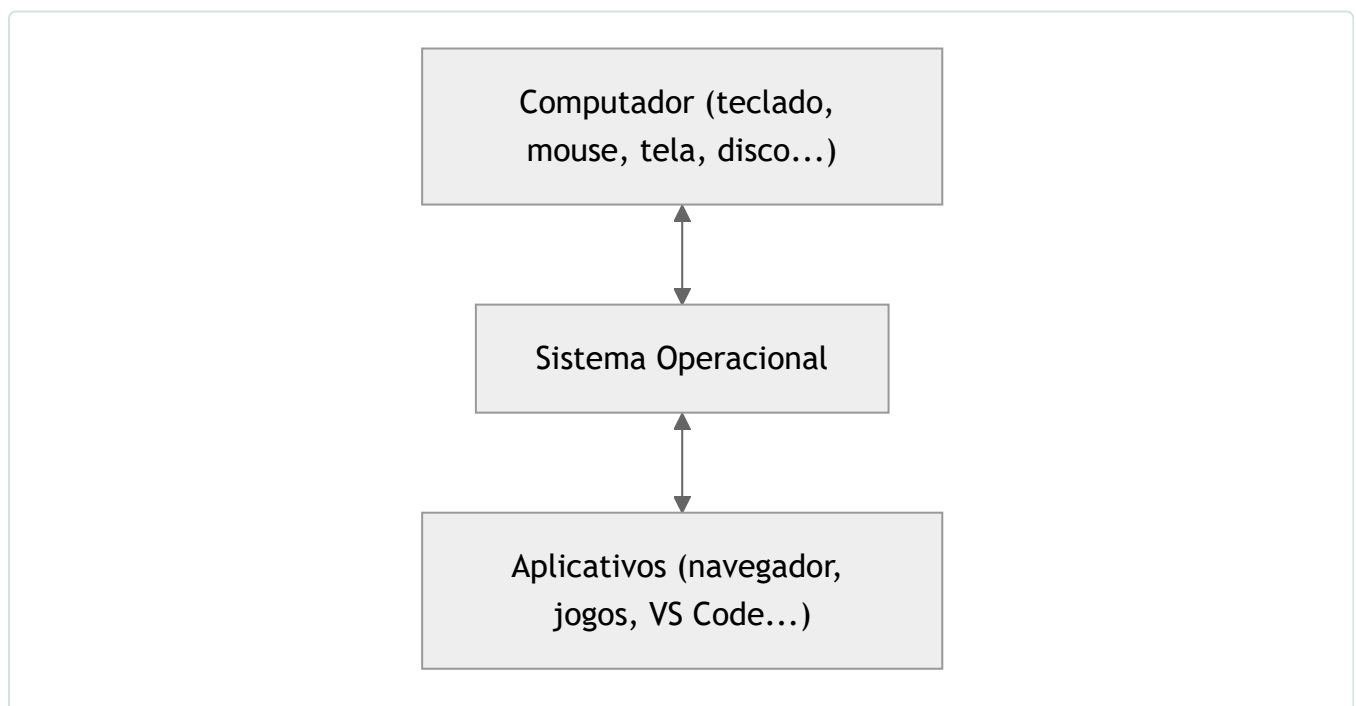
- o que é um **sistema operacional**; **Windows** e **Linux**
- o que são **arquivos**, **pastas**, **caminho** e **extensão**
- como usar o **terminal** para navegar e criar pastas
- as ferramentas do curso: **Python 3.12** (a linguagem), **uv**, **VS Code** e o pacote **pgzero**
- o que é um **ambiente virtual (venv)**

Sistema operacional (SO)

É o programa principal que gerencia o computador — abre outros programas, guarda seus arquivos, controla o teclado e o mouse. Quando você liga o computador, é o SO que aparece.

Windows e **Linux** são dois exemplos de SO. O curso funciona igual nos dois — só muda a aparência das janelas e alguns comandos de instalação. Só precisa saber **qual SO o seu computador usa**.

Nenhum aplicativo fala direto com o computador — ele sempre passa pelo SO:



Arquivos, pastas, caminho e extensão

- **Arquivo:** uma coisa guardada no computador (um texto, uma foto, um programa).
- **Pasta:** uma "caixa" que guarda arquivos, pra organizar.
- **Caminho (path):** o "endereço" de um arquivo ou pasta — o caminho que você percorre, pasta por pasta, até chegar nele. Igual o endereço de uma casa.
- **Extensão:** a partezinha depois do ponto no nome do arquivo (`.png` , `.py`) — diz que tipo de arquivo é. Os arquivos de Python terminam em `.py` .

Terminal (CLI): usando o computador por comandos

Além de clicar nos ícones, dá pra usar o computador **digitando comandos**. Isso se chama **terminal** (ou CLI). Hoje a gente instala quase tudo por aqui.

Como abrir:

- **Windows:** menu Iniciar, digite "PowerShell" e aperte Enter.
- **Linux:** aperte `Ctrl+Alt+T` , ou procure "Terminal" no menu.

Comandos básicos:

O que eu quero fazer	Comando	O que deve aparecer
Ver em qual pasta eu estou	<code>pwd</code>	o caminho completo da pasta atual
Ver o que tem dentro da pasta atual	<code>dir</code> (Windows) ou <code>ls</code> (Linux)	a lista de arquivos e pastas dali
Entrar numa pasta	<code>cd nome-da-pasta</code>	nada; confira com <code>pwd</code>
Voltar uma pasta	<code>cd ..</code>	nada; confira com <code>pwd</code>
Criar uma pasta nova	<code>mkdir nome-da-pasta</code>	nada; a pasta aparece no <code>dir / ls</code>
Remover uma pasta vazia	<code>rmdir nome-da-pasta</code>	nada; a pasta some do <code>dir / ls</code>

No terminal, **"deu certo" quase sempre é "não apareceu nada"** — a gente confere o efeito com `pwd` ou `dir / ls` . Se aparecer uma linha de erro, leia o que ela diz.

As ferramentas do curso

Ferramenta	Para que serve
Python 3.12	A linguagem do curso — é ela que a gente vem aprender. A versão é 3.12 de propósito (garante o <code>pygame</code> pronto).
uv	O instalador do curso: instala o Python, cria o ambiente virtual e instala os pacotes, tudo com os mesmos comandos. Como é ele que traz o Python, é a primeira coisa a rodar.
VS Code + extensão Python	A IDE : junta editor, terminal e mensagens de erro num lugar só. O VS Code não é uma linguagem — é onde a gente escreve e roda o código.
pgzero (Pygame Zero)	Um pacote pronto, baixado de graça do PyPI , que dá os recursos de janela e desenho do jogo. Instalado dentro do ambiente virtual.

Passo a passo (siga na ordem)

1. Ver o que já está instalado

No terminal:

```
python --version
uv --version
code --version
```

O que deve aparecer: para o que já está instalado, uma linha com a versão (ex.: Python 3.12.4). Para o que falta, um erro tipo `'python' não é reconhecido / command not found` — normal, é só instalar abaixo.

2. Instalar o Python 3.12 (com o uv)

O curso é de Python — mas quem instala o Python pra gente é o `uv` , então ele vem primeiro.

- Instalar o `uv` :
- **Windows:** `powershell -c "irm https://astral.sh/uv/install.ps1 | iex"`
- **Linux:** `curl -LsSf https://astral.sh/uv/install.sh | sh`
- Deve terminar com uma mensagem de sucesso do instalador.
- **Feche e abra o terminal de novo** e confirme: `uv --version` → deve imprimir algo como `uv 0.4.20` .
- Instalar o Python 3.12: `uv python install 3.12` → baixa (barra de progresso) e termina com `Installed Python 3.12.x` , sem erro.
- Confirmar: `uv python list` → deve ter uma linha com `3.12` e um caminho ao lado.

3. Instalar o VS Code e a extensão Python

- Instalar o VS Code:
- **Windows:** `winget install -e --id Microsoft.VisualStudioCode`
- **Linux:** `sudo snap install code --classic`
- Deve mostrar o progresso e terminar com sucesso.
- **Feche e abra o terminal de novo** e confirme: `code --version` → responde com três linhas.
- Abra o VS Code, clique no ícone de extensões (`Ctrl+Shift+X`), busque **Python** (da Microsoft) e clique em **Install** → o botão passa a mostrar "Uninstall".

Se o `winget / snap` não funcionar, baixe o VS Code em <https://code.visualstudio.com>.

4. Criar a pasta do curso

No terminal, vá até onde a pasta vai ficar (ex.: a Área de Trabalho) e crie a estrutura:

```
mkdir crescendo-como-jesus
cd crescendo-como-jesus
mkdir aula-01
pwd
```

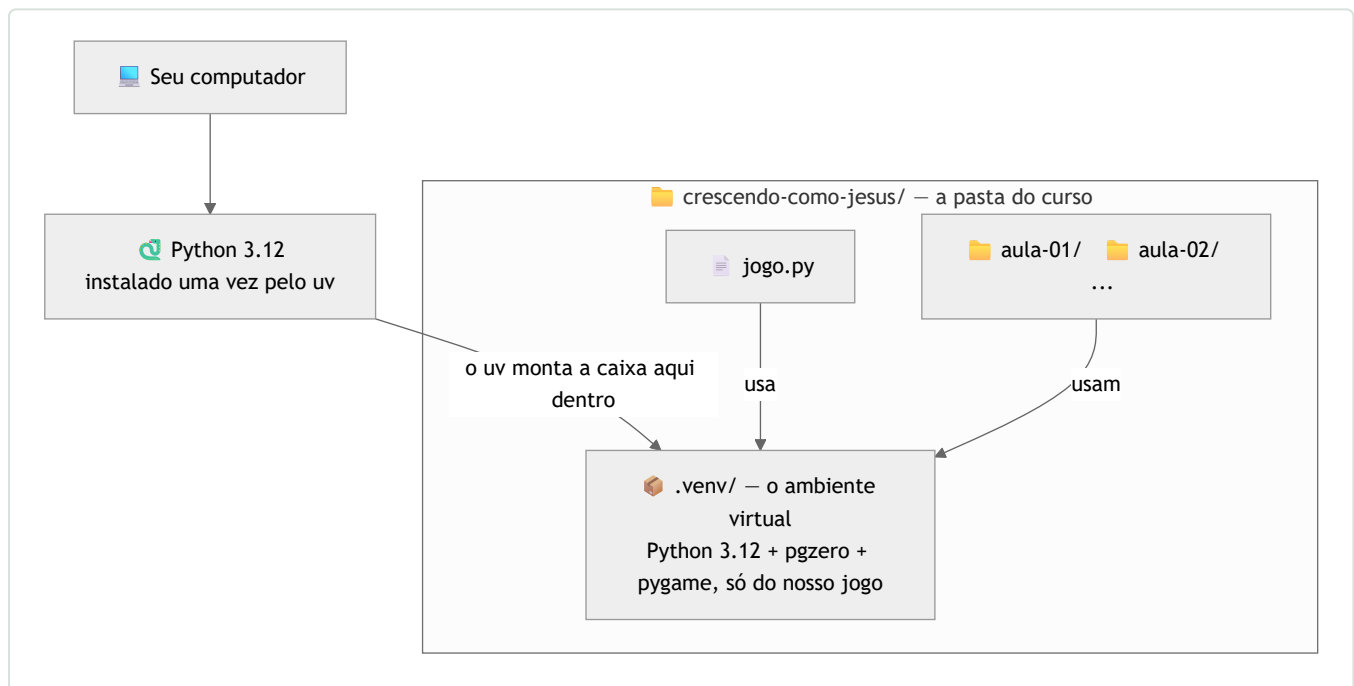
O que deve aparecer: os três primeiros comandos não respondem nada (normal); o `pwd` no fim mostra o caminho completo terminando em `crescendo-como-jesus`. Um `dir / ls` mostra a `aula-01` lá dentro.

- `crescendo-como-jesus/` é a **pasta principal** do curso — é aqui que tudo vai acontecer.
- `aula-01/` guarda os testes soltos da primeira aula. Cada aula terá a sua (`aula-02`, ...).
- Use `aula-01` (com traço), **não** `modulo-01`.
- O arquivo `jogo.py` vai ficar na **raiz** de `crescendo-como-jesus/`, mas ele só nasce no Módulo 1.

Abra essa pasta no VS Code: ainda dentro dela no terminal, digite `code .` → o VS Code abre com `CRESCENDO-COMO-JESUS` no topo do explorador da lateral e `aula-01` listada dentro.

5. Criar o ambiente virtual (venv)

O **venv** é uma "caixa" separada com o Python e os pacotes só do nosso projeto — assim o ambiente fica igual em qualquer computador. Essa caixa mora **dentro** da pasta do curso, na subpasta `.venv/`, e é usada tanto pelo `jogo.py` quanto por todas as pastas `aula-NN/`:



Com a pasta aberta no VS Code, abra o **terminal integrado** (`Ctrl+``) e rode:

```
uv venv --python 3.12 .venv
uv pip install pgzero
uv pip freeze > requirements.txt
```

O que deve aparecer:

- `uv venv ...` → `Creating virtual environment at: .venv`; a pasta `.venv` surge no explorador.

- `uv pip install pgzero` → a lista dos pacotes (`pgzero`, `pygame`, `numpy` ...) e `Installed N packages`, sem linha vermelha.
- `uv pip freeze > requirements.txt` → nada no terminal; o arquivo `requirements.txt` aparece no explorador.

Depois selecione o interpretador: `Ctrl+Shift+P` → **Python: Select Interpreter** → escolha o que tem `.venv` (costuma vir como "Recommended") → o canto inferior direito do VS Code passa a mostrar `3.12.x ('.venv')`.

6. Testar se está tudo pronto

Isto **não é** aula de Python — é só pra ver se tudo liga.

- No explorador do VS Code, dentro de `aula-01/`, crie o arquivo `teste.py` com: `python print("ambiente pronto para a aula")`
- Clique no botão ► **Run Python File** (canto superior direito).
- **O que deve aparecer:** no terminal, a linha exata `ambiente pronto para a aula`, sem erro. Se apareceu isso, **está tudo pronto!** (Pode apagar o `teste.py` depois.)

Checklist final

- [] `python --version` (ou `uv python list` com `3.12`), `uv --version` e `code --version` respondem
- [] extensão **Python** instalada no VS Code
- [] pasta `crescendo-como-jesus/` criada, com `aula-01/` dentro
- [] `.venv` criado na raiz de `crescendo-como-jesus/`, `pgzero` instalado, `requirements.txt` gerado
- [] interpretador `.venv` selecionado no VS Code
- [] `teste.py` rodou e imprimiu a frase

Quando tudo estiver marcado, seu computador está pronto para o Módulo 1 — é lá que o `jogo.py` começa.

Palavras novas

Termo	O que significa
Sistema operacional (SO)	O programa principal que gerencia o computador
Windows / Linux	Dois exemplos de sistema operacional
Arquivo / Pasta	Uma coisa guardada no computador / uma "caixa" que organiza arquivos
Extensão	A parte depois do ponto no nome do arquivo (ex.: <code>.py</code>)
Caminho (path)	O endereço completo até um arquivo ou pasta
Terminal / CLI	Usar o computador digitando comandos
<code>cd</code> / <code>mkdir</code> / <code>rmdir</code>	Andar entre pastas / criar pasta / remover pasta vazia
<code>code .</code>	Abre o VS Code na pasta atual do terminal
IDE / VS Code	Programa que junta editor, execução e erros / a IDE do curso
Python 3.12	A linguagem que vamos usar; versão fixa de propósito
<code>uv</code>	Instala o Python, cria o venv e instala os pacotes
Pacote / PyPI	Código pronto e gratuito / o repositório de onde ele vem
<code>pygame</code> (Pygame Zero)	O pacote que dá janela e desenho pro jogo
Ambiente virtual (venv)	"Caixa" isolada com o Python e os pacotes do projeto
<code>requirements.txt</code>	Lista das versões exatas dos pacotes, pra recriar o ambiente igual

Deu dúvida? Tenta isso primeiro

- `uv` / `code` **não reconhecido depois de instalar:** feche e abra o terminal de novo.
- `winget` **não funciona:** baixe o VS Code em <https://code.visualstudio.com>.
- `uv python install 3.12` **parece travado:** ele baixa o Python na primeira vez — aguarde.
- **Me perdi entre as pastas:** use `pwd`, e a barra de endereço do gerenciador de arquivos.
- `rmdir` **deu erro:** a pasta não está vazia (confira com `dir` / `ls`).
- **VS Code não mostra o `.venv`:** confirme que a pasta `.venv` está **dentro** de `crescendo-como-jesus/` e recarregue a janela (`Ctrl+Shift+P` → "Developer: Reload Window").
- `teste.py` **deu erro:** repita o passo 5 (Select Interpreter → `.venv`) e confira o interpretador no canto inferior direito do VS Code.

Módulo 1 — Conhecendo a programação (material do aluno)

O jogo que vamos construir

Ao longo do curso vamos construir, um pedaço por aula, um **jogo de labirinto no estilo Pac-Man** com a ideia do versículo:

"E crescia Jesus em sabedoria, e em estatura, e em graça, para com Deus e os homens." (Lucas 2:52)

No jogo, o personagem anda pelo "labirinto da vida":

- coletando **bons hábitos** — 🙏 oração, 📖 Bíblia, 🏛️ culto, ❤️ obedecer aos pais, 🤝 ajudar o próximo —, que aumentam a **estatura**;
- desviando das **tentações** — 😡 desobediência, 😬 mentira, 😴 preguiça, 🎮 distração —, que fazem perder uma vida;
- a **oração** é um poder: por alguns segundos as tentações fogem e podem ser vencidas;
- são **4 fases**: Sabedoria → Estatura → Graça com Deus → Graça com os homens.

O professor vai mostrar o **jogo pronto rodando** — é para onde a gente vai chegar. Hoje vamos fazer só o começo: a **primeira tela**.

O que vamos aprender hoje

- O que é um **programa** e um **algoritmo**
- Como o computador executa as instruções: **uma de cada vez, na ordem** em que foram escritas
- O que é uma **biblioteca** e de onde vem o código do **Pygame Zero**
- O que é uma **variável**
- O que é uma **função** e por que o **recuo** (indentação) importa tanto no Python
- Escrever e rodar o nosso **primeiro programa** em Python, com o **Pygame Zero**

O que você já tem pronto (do Módulo 0)

Hoje não instala nada. No Módulo 0 sua máquina já ficou pronta:

- Python 3.12 e o VS Code (com a extensão Python);
- a pasta `crescendo-como-jesus/` com a subpasta `aula-01/`;
- o ambiente virtual `.venv/` dentro dela, com o pacote `pgzero` instalado;
- o interpretador `.venv` selecionado no VS Code.

Confira uma coisa só: no canto inferior direito do VS Code deve aparecer algo como `3.12.x ('.venv')`. Se não aparecer, aperte `Ctrl+Shift+P`, digite **Python: Select Interpreter** e escolha o que tem `.venv` (costuma vir marcado como "Recommended"). Esse é o "Python do projeto" — o que tem o `pgzero` que você instalou no Módulo 0.

Programa, algoritmo e a ordem das coisas

- Um **programa** é uma **lista de instruções** que o computador segue.
- Um **algoritmo** é a **sequência de passos** para resolver um problema — como uma **receita de bolo**: cada passo na ordem certa.
- O computador faz **uma instrução de cada vez, de cima para baixo**, na ordem em que você escreveu. Ele **não adivinha** o que você quis dizer nem pula na frente. Numa receita, trocar "leve ao forno" com "misture os ovos" estraga tudo — no código é igual.

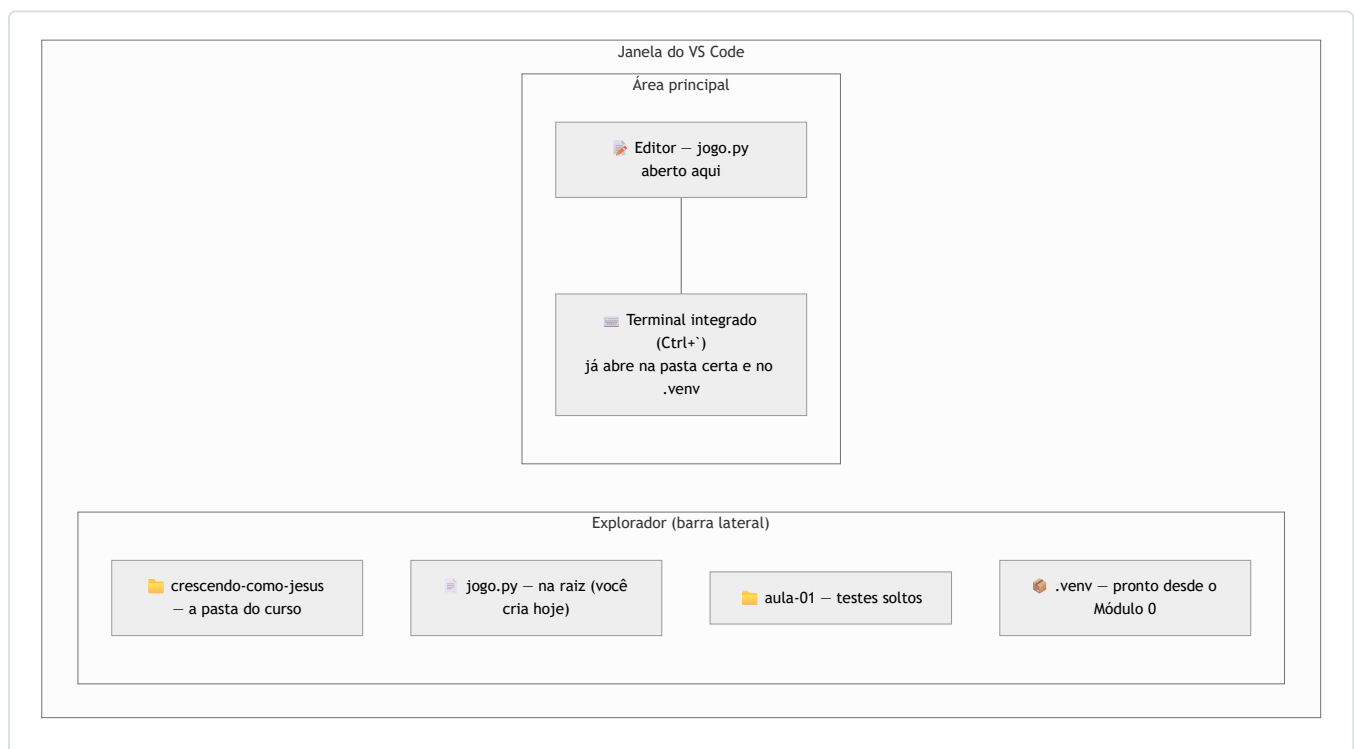
Abrindo a pasta do curso no VS Code

Abra a pasta `crescendo-como-jesus/` (a que você criou no Módulo 0):

- no terminal, **dentro dela**, digite `code .` — ou
- no VS Code, **File > Open Folder...** e escolha a pasta.

No explorador (barra lateral) você vê `CRESCENDO-COMO-JESUS` no topo, com `aula-01/` e `.venv/` dentro.

O arquivo que vamos criar hoje, o `jogo.py`, fica na **raiz** de `crescendo-como-jesus/` — **não** dentro de `aula-01/`. Ele é o arquivo que vai crescer o curso inteiro. A `aula-01/` é só para testes soltos, quando você quiser experimentar algo fora do jogo.



Como o Python roda um arquivo

Você é quem diz qual arquivo o Python vai rodar. Ele pega esse arquivo e executa **do começo ao fim, linha por linha**, na ordem em que as linhas aparecem.

Dois jeitos de rodar, que fazem **exatamente a mesma coisa**:

- **Botão ▶ "Run Python File"** (canto superior direito do VS Code): roda o arquivo que está aberto no editor.

- **Pelo terminal integrado** (Ctrl+` para abrir): `python jogo.py`
- `python` — o programa que **lê e executa** o código.
- `jogo.py` — o arquivo que ele vai ler, começando pela primeira linha.

O botão é só um atalho para não digitar isso toda vez.

Construindo a primeira tela, passo a passo

Crie um arquivo novo e salve como `jogo.py` na **raiz** de `crescendo-como-jesus/`. Vamos escrever o código aos poucos e **rodar depois de cada passo** — assim dá para ver o efeito de cada pedaço antes de seguir.

Passo 1 — o esqueleto mínimo

```
import pgzrun

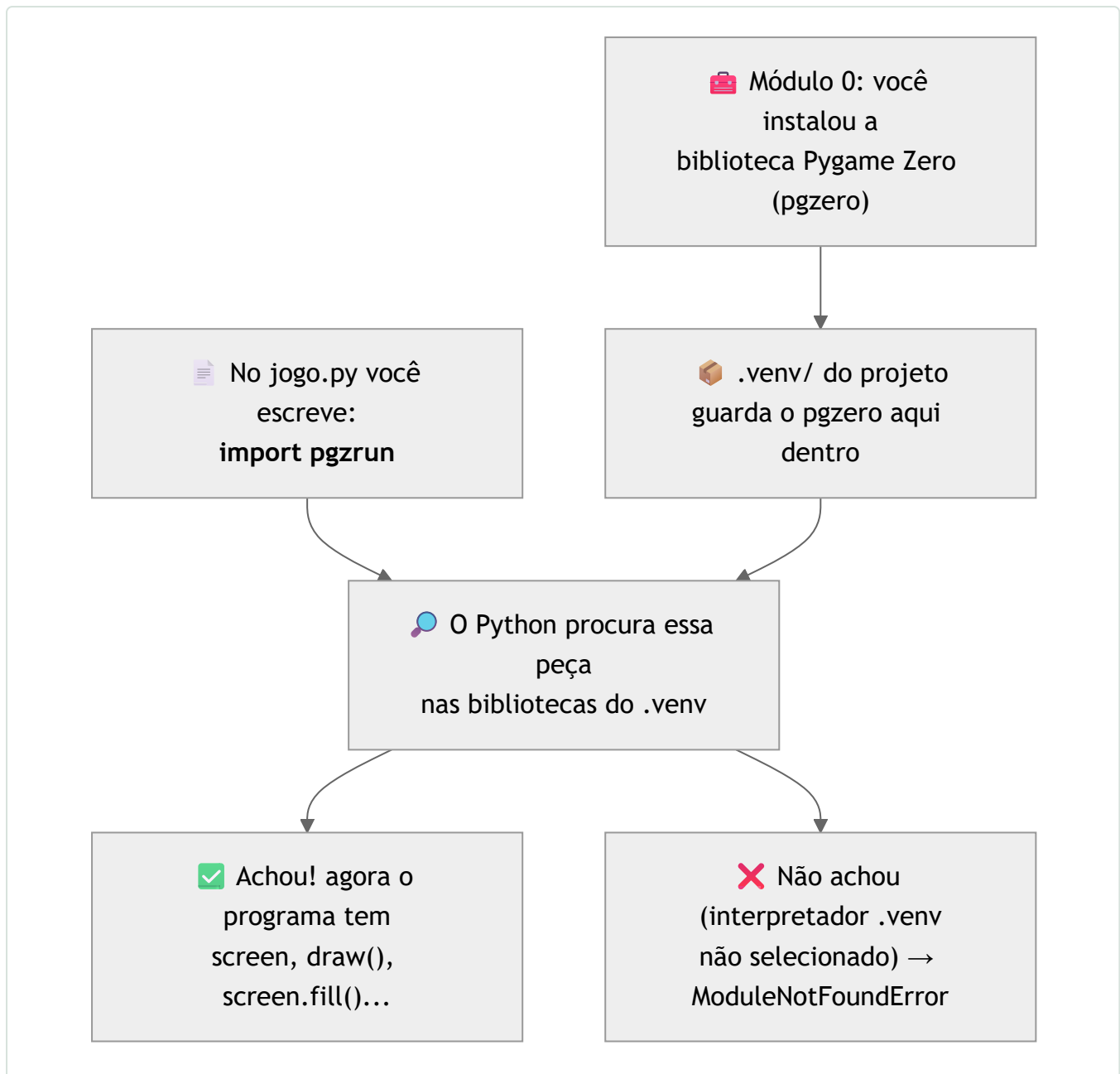
pgzrun.go()
```

- `import pgzrun` — **traz** para o seu programa um código pronto que veio **de fora**. É ele que dá `screen`, `draw()` e o resto do jogo. Vai sempre na **primeira** linha.
- `pgzrun.go()` — **liga o motor do jogo**: abre a janela e fica repetindo "desenha a tela, escuta o teclado, desenha de novo...". Vai sempre na **última** linha.

De onde vem o `pgzrun`?

Você não escreveu esse código, e ele não estava na sua máquina antes do curso. Ele vem de uma **biblioteca** (também chamada **pacote**): um monte de código pronto que outras pessoas escreveram e deixaram disponível para todo mundo usar — assim você não precisa fazer tudo do zero.

- A biblioteca que usamos se chama **Pygame Zero**. No **Módulo 0**, ela foi baixada da internet e guardada **dentro do** `.venv/` do seu projeto.
- Repare numa coisa: você instalou o `pgzero`, mas escreve `import pgzrun`. O pacote inteiro é o `pgzero`; o `pgzrun` é uma **peça de dentro dele** — a "chave de ligar" que abre a janela do jogo.
- Quando o Python lê `import pgzrun`, ele vai **procurar** essa peça nas bibliotecas do `.venv`. Se acha, o seu programa ganha o `screen`, o `draw()`, o `screen.fill()` e o resto. Se não acha (interpretador errado), aparece `ModuleNotFoundError: No module named 'pgzero'`.



Rode. Deve abrir uma **janelinha preta**, do tamanho padrão — já é um jogo Pygame Zero de verdade, só que vazio. Feche a janela para continuar.

Se aparecer o erro `screen is not defined`, é porque falta uma dessas duas linhas (`import pgzrun` ou `pgzrun.go()`). Se aparecer `ModuleNotFoundError`, é o interpretador `.venv` que não está selecionado.

Passo 2 — tamanho e título da janela

Adicione estas três linhas **entre** o `import` e o `pgzrun.go()`:

```
WIDTH = 800
HEIGHT = 600
TITLE = "Crescendo como Jesus"
```

- Isto é uma **variável**: um **nome que guarda um valor**. `WIDTH = 800` quer dizer "de agora em diante, `WIDTH` vale 800".
- `WIDTH` (largura) e `HEIGHT` (altura) são o tamanho da janela **em pixels** — os pontinhos que formam a tela.
- `TITLE` é um **texto** (fica entre aspas) e aparece na **barra** da janela.
- O Pygame Zero reconhece **esses três nomes sozinho** — por isso eles precisam ser escritos assim mesmo, em letras maiúsculas.

Rode de novo. A janela nasce do tamanho `800 x 600` e com o título na barra.

Passo 3 — pintar o fundo

```
COR_DE_FUNDO = "skyblue"

def draw():
    screen.fill(COR_DE_FUNDO)
```

Antes de rodar, repare em duas coisas novas:

- **Função:** é um pedaço de código com **nome**, guardado para ser executado quando alguém chama. `def` cria a função, o nome (`draw`) vem logo depois, e os `()` e `:` no fim são obrigatórios. A função `draw()` é **especial: o próprio Pygame Zero chama ela sozinho**, várias vezes por segundo, sempre que precisa redesenhar a tela. Você só escreve **o que** desenhar dentro dela.
- **Indentação (o recuo):** a linha `screen.fill(...)` **não começa colada na margem** — está recuada (um "tab" para dentro). No Python, esse recuo **não é enfeite, é regra**: é ele que diz "esta linha faz parte da função `draw`". Se faltar ou ficar torto, dá um erro chamado `IndentationError`. O VS Code ajuda: depois de uma linha que termina em `:`, ele recua a próxima sozinho quando você aperta Enter.

`screen.fill(COR_DE_FUNDO)` **pinta a tela inteira** com a cor. `COR_DE_FUNDO` é só uma variável com um texto dentro.

Rode de novo. A janela aparece pintada. Troque `"skyblue"` por outro nome de cor (`"darkgreen"`, `"black"`, `"white"` ...) e rode outra vez para ver a diferença. A lista completa de nomes está na [documentação do Pygame Zero](#); também dá para usar um valor RGB, como `(255, 0, 0)` para vermelho.

Passo 4 — a mensagem de boas-vindas

Dentro de `draw()`, **depois** do `screen.fill`, use `screen.draw.text(...)`:

```
COR_DO_TEXTO = "white"

def draw():
    screen.fill(COR_DE_FUNDO)
    screen.draw.text(
        "Bem-vindo(a) ao " + TITLE + "!",
        center=(WIDTH // 2, HEIGHT // 2),
        fontsize=40,
        color=COR_DO_TEXTO,
    )
```

Cada parte de `screen.draw.text(...)`:

- `"Bem-vindo(a) ao " + TITLE + "!"` — o **texto** que vai na tela. O `+` **junta textos**: o pedaço fixo `"Bem-vindo(a) ao "`, mais o conteúdo da variável `TITLE`, mais `!"`.
- `center=(WIDTH // 2, HEIGHT // 2)` — o ponto onde o texto fica **centralizado**. O `//` é **divisão inteira**: `WIDTH // 2` é a metade da largura, ou seja, o **meio da tela**.
- `fontsize=40` — o **tamanho** da letra.
- `color=COR_DO_TEXTO` — a **cor** do texto (mesma ideia de variável do passo 3).

Rode de novo. A mensagem aparece no centro da tela.

Passo 5 — o seu nome como pecador

Mais uma chamada de `screen.draw.text`, um pouco **mais abaixo** na tela — repare no `+ 60`:

```
NOME_DO_PECADOR = "Escreva seu nome aqui"

def draw():
    screen.fill(COR_DE_FUNDO)
    screen.draw.text(
        "Bem-vindo(a) ao " + TITLE + "!",
        center=(WIDTH // 2, HEIGHT // 2),
        fontsize=40,
        color=COR_DO_TEXTO,
    )
    screen.draw.text(
        "Pecador: " + NOME_DO_PECADOR,
        center=(WIDTH // 2, HEIGHT // 2 + 60),
        fontsize=24,
        color=COR_DO_TEXTO,
    )
```

- O `+ 60` no `center` empurra este texto **60 pixels para baixo**, para não ficar em cima do texto anterior.

Rode uma última vez e troque `"Escreva seu nome aqui"` pelo seu nome de verdade.

Pronto — sua tela já tem tudo da missão de hoje! Compare com o [exemplo.py](#) deste módulo: ele vem com **um passo a mais** (o versículo de Lucas 2:52 na tela), que é exatamente o desafio extra logo abaixo.

Sua missão

Crie uma tela com:

- [] título do jogo (aparece na barra da janela);
- [] uma mensagem de boas-vindas na tela;
- [] uma cor de fundo escolhida por você;
- [] seu nome como pecador, escrito na tela.

Construa **do zero**, seguindo os passos acima — não copie o `exemplo.py` pronto; ele é só para comparar no fim ou para destravar se você empacar.

Desafios

- **Desafio extra:** adicione uma segunda mensagem na tela — que tal o versículo de Lucas 2:52?
- **Desafio criativo:** escolha cores que combinem com o jogo que você imagina criar.

Palavras novas

Termo	O que significa
Programa	Uma lista de instruções que o computador segue
Algoritmo	A sequência de passos para resolver um problema
Sequência de execução	O computador faz uma instrução de cada vez, de cima para baixo, na ordem
Python	A linguagem que usamos para escrever os programas
VS Code	A IDE (o programa) onde escrevemos e rodamos o código
Executar (Run)	Mandar o computador seguir as instruções do programa
<code>import</code>	Trazer para o nosso programa um código pronto de fora (uma biblioteca)
Biblioteca / pacote	Um monte de código pronto que outras pessoas escreveram para todo mundo usar
<code>pgzero</code> (Pygame Zero)	A biblioteca que dá <code>screen</code> , <code>draw()</code> e o resto do jogo — instalada no <code>.venv</code> no Módulo 0
<code>pgzrun</code>	A peça de dentro do <code>pgzero</code> que liga o jogo (<code>import pgzrun</code> no topo, <code>pgzrun.go()</code> no fim)
Variável	Um nome que guarda um valor (<code>WIDTH = 800</code>)
<code>WIDTH</code> / <code>HEIGHT</code> / <code>TITLE</code>	Nomes que o Pygame Zero reconhece: largura, altura e título da janela
Função	Um pedaço de código com nome, executado quando alguém chama (criada com <code>def</code>)
<code>draw()</code>	Função especial que o Pygame Zero chama sozinho para desenhar a tela
Indentação (recuo)	O espaço no começo da linha que diz o que "pertence" a uma função — no Python é regra, não enfeite
<code>screen.fill(cor)</code>	Pinta a tela inteira de uma cor
<code>screen.draw.text(...)</code>	Desenha um texto na tela (posição, tamanho e cor)
<code>//</code>	Divisão inteira — <code>WIDTH // 2</code> é a metade da largura (o meio da tela)

Deu erro? Tenta isso primeiro

- `screen is not defined`: confira se o arquivo **começa** com `import pgzrun` e **termina** com `pgzrun.go()`.
- `ModuleNotFoundError: No module named 'pgzero'`: o interpretador certo não está selecionado — `Ctrl+Shift+P` → **Python: Select Interpreter** → o que tem `.venv`. (Você não precisa instalar nada; isso já foi feito no Módulo 0.)
- `IndentationError`: alguma linha dentro de `draw()` não está recuada igual às outras — confira se todas têm o mesmo espaço antes do código.
- **Erro apontando uma linha**: leia a mensagem — ela sempre mostra **em qual linha** está o problema.

- **Cor não funciona:** confira o nome na lista de cores do Pygame Zero, ou use um valor RGB como `(255, 0, 0)`.

Ao final, **salve** o `jogo.py` (na raiz de `crescendo-como-jesus/`) — vamos usá-lo em todos os módulos!

Módulo 2 — O personagem e a primeira decisão (material do aluno)

A ideia de hoje

No Módulo 1 seu `jogo.py` ganhou uma tela de boas-vindas. Hoje ela vai virar a **primeira cena de verdade** do jogo: um personagem aparece na tela, uma pergunta sobre um hábito cristão é feita (ex.: "Você orou hoje?"), e o jogador responde clicando em **Sim** ou **Não**. O personagem **reage**: cresce e muda de cor no "Sim", fica do mesmo jeito (ou menor) no "Não" — a primeira "decisão" programada do jogo.

O que vamos aprender hoje

- **Coordenadas**: como dizer onde algo fica na tela — um par de números `(x, y)`
- Desenhar formas: um **círculo** (o personagem) e **retângulos** (os botões)
- **Variáveis lógicas** (`True` / `False`): guardar se algo já aconteceu ou não
- `if` / `elif`: fazer o programa decidir o que fazer, comparando com `==`
- **Evento de clique**: uma função especial que o Pygame Zero chama sozinho quando você clica na tela
- `global`: como uma função consegue mudar uma variável que existe fora dela

O que você já tem pronto (do Módulo 1)

Abra de novo o `jogo.py` que você criou no Módulo 1, na raiz de `crescendo-como-jesus/`. Ele já tem:

```
import pgzrun

WIDTH = 800
HEIGHT = 600
TITLE = "Crescendo como Jesus"

COR_DE_FUNDO = "skyblue"
COR_DO_TEXTO = "white"
NOME_DO_PECADOR = "... "

def draw():
    screen.fill(COR_DE_FUNDO)
    screen.draw.text(...)
    screen.draw.text(...)

pgzrun.go()
```

Hoje vamos **trocar o conteúdo de dentro de** `draw()` pela cena do personagem — o `import`, o `WIDTH` / `HEIGHT` / `TITLE` e o `pgzrun.go()` continuam exatamente iguais, no começo e no fim do arquivo.

A mensagem de boas-vindas e o versículo saem de cena, mas o seu **nome de pecador** continua na tela — só muda de lugar, para um cantinho discreto.

Construindo o personagem e a decisão, passo a passo

Vamos escrever aos poucos e **rodar depois de cada passo**, do mesmo jeito do Módulo 1.

Passo 1 — coordenadas e o personagem parado

Antes da função `draw()` (fora dela), crie as variáveis do personagem:

```
# Posição e tamanho do personagem (um círculo, por enquanto)
personagem_x = WIDTH // 2
personagem_y = HEIGHT // 2 - 60
personagem_raio = 40
personagem_cor = "gold"
```

Troque o conteúdo de `draw()` para desenhar o círculo no lugar do texto de boas-vindas — mas **mantenha** o nome do pecador, só que agora **num cantinho** da tela em vez do centro:

```
def draw():
    screen.fill(COR_DE_FUNDO)

    screen.draw.text(
        "Pecador: " + NOME_DO_PECADOR,
        topleft=(10, HEIGHT - 30),
        fontsize=16,
        color=COR_DO_TEXTO,
    )

    screen.draw.filled_circle(
        (personagem_x, personagem_y), personagem_raio, personagem_cor
    )
```

- **Coordenada:** todo lugar na tela é um par de números `(x, y)`. `x` cresce **para a direita**, `y` cresce **para baixo** (o `(0, 0)` é o canto **superior esquerdo** da janela — diferente do caderno de matemática, onde `y` cresce para cima!).
- `WIDTH // 2` e `HEIGHT // 2` são o **meio da tela** (já usado no Módulo 1); o `- 60` sobe o personagem um pouco, para deixar espaço pra pergunta em cima.
- `screen.draw.filled_circle((x, y), raio, cor)` desenha um **círculo preenchido**: centro em `(x, y)`, tamanho `raio`, na cor indicada.
- `topleft=(10, HEIGHT - 30)` é outro jeito de posicionar texto: em vez de `center` (que centraliza), `topleft` ancora o **canto superior esquerdo** do texto naquele ponto — aqui, 10 pixels da borda esquerda e perto do rodapé (`HEIGHT - 30`).

Rode. No canto inferior esquerdo aparece "Pecador: [seu nome]", pequeno. No meio da tela, um pouco acima do centro, aparece uma **bola dourada**, sobre o fundo azul-claro.

Passo 2 — a pergunta na tela

Junto das outras variáveis, no topo do arquivo:

```
PERGUNTA = "Você orou hoje?"
```

Dentro de `draw()`, **antes** do círculo, desenhe o texto da pergunta:

```
def draw():
    screen.fill(COR_DE_FUNDO)

    screen.draw.text(
        PERGUNTA,
        center=(WIDTH // 2, HEIGHT // 2 - 160),
        fontsize=32,
        color=COR_DO_TEXTO,
    )

    screen.draw.filled_circle(
        (personagem_x, personagem_y), personagem_raio, personagem_cor
    )
```

`screen.draw.text(...)` já apareceu no Módulo 1 — só muda a posição (`HEIGHT // 2 - 160`, bem acima do personagem).

Rode. Agora a pergunta **"Você orou hoje?"** aparece em cima da bola dourada.

Passo 3 — os botões Sim e Não

Junto das outras variáveis, no topo:

```
# Área (retângulo) de cada botão: (x, y, largura, altura)
botao_sim = Rect((WIDTH // 2 - 160, HEIGHT // 2 + 120), (120, 50))
botao_nao = Rect((WIDTH // 2 + 40, HEIGHT // 2 + 120), (120, 50))
```

- `Rect((x, y), (largura, altura))` é um **retângulo**: `(x, y)` é o canto **superior esquerdo**, e `(largura, altura)` é o tamanho. O Pygame Zero já sabe o que é `Rect` — não precisa de `import` extra.
- Cada botão é só uma **área invisível** por enquanto — o próximo passo desenha ela.

Dentro de `draw()`, depois do personagem, desenha os dois botões:

```
screen.draw.filled_rect(botao_sim, "green")
screen.draw.text("Sim", center=botao_sim.center, fontsize=28, color="white")

screen.draw.filled_rect(botao_nao, "red")
screen.draw.text("Não", center=botao_nao.center, fontsize=28, color="white")
```

- `screen.draw.filled_rect(retangulo, cor)` pinta o retângulo.
- `botao_sim.center` é um **atalho do próprio `Rect`**: ele calcula sozinho o ponto central do retângulo — não precisamos fazer conta.

Rode. Aparecem um botão **verde "Sim"** e um botão **vermelho "Não"**, abaixo do personagem. Clicar ainda não faz nada — isso é o próximo passo.

Passo 4 — detectando o clique

Depois de `draw()` (fora dela), crie uma nova função:

```
def on_mouse_down(pos):
    if botao_sim.collidepoint(pos):
        print("cliqueu no Sim")
    elif botao_nao.collidepoint(pos):
        print("cliqueu no Não")
```

- `on_mouse_down(pos)` é uma **função especial**, igual `draw()`: o Pygame Zero **chama ela sozinho** toda vez que você clica na janela, e entrega em `pos` a coordenada `(x, y)` de onde foi o clique.
- `retangulo.collidepoint(pos)` responde `True` ou `False`: "esse ponto está **dentro** deste retângulo?".
- `if / elif` compara: **se** o clique caiu dentro de `botao_sim`, faz uma coisa; **senão, se** caiu dentro de `botao_nao`, faz outra. Repare no `==` que não aparece aqui — `collidepoint` já devolve `True / False` pronto, então o `if` testa ele direto.

Rode e clique nos dois botões. Nada muda **na tela**, mas abra o **terminal integrado** (`Ctrl+``): a cada clique dentro de um botão aparece a mensagem `cliqueu no Sim` ou `cliqueu no Não`. Clicar **fora** dos dois botões não imprime nada. Feche a janela para continuar.

Passo 5 — guardando a resposta e fazendo o personagem reagir

Troque os `print(...)` do passo anterior por código que **muda o personagem de verdade**. Junto das outras variáveis, no topo do arquivo:

```
# Estado da resposta: começa sem nenhuma resposta ainda
respondeu = False
resposta_foi_sim = False
```

- `True / False` são os dois únicos valores **lógicos** do Python — aqui guardam "o jogador já respondeu?" e "a resposta foi Sim?".

Agora reescreva `on_mouse_down`:

```
def on_mouse_down(pos):
    global respondeu, resposta_foi_sim, personagem_raio, personagem_cor

    if botao_sim.collidepoint(pos):
        respondeu = True
        resposta_foi_sim = True
        personagem_raio = 60
        personagem_cor = "yellow"
    elif botao_nao.collidepoint(pos):
        respondeu = True
        resposta_foi_sim = False
        personagem_raio = 30
        personagem_cor = "gray"
```

- **Por que o `global`?** Uma função, por padrão, **não pode mudar** uma variável de fora dela — só ler. A linha `global respondeu, resposta_foi_sim, personagem_raio, personagem_cor` avisa o Python: "estas variáveis não são novas, são as que já existem lá em cima — quero **mudar elas de verdade**". Sem o `global`, o Python cria uma cópia local que some assim que a função termina, e a tela nunca muda.
- No "Sim": `personagem_raio` fica maior (60) e a cor muda para `"yellow"`.
- No "Não": `personagem_raio` fica menor (30) e a cor muda para `"gray"`.

Rode e clique no "Sim". O personagem **cresce e fica amarelo**. Feche, rode de novo e clique no **"Não"**: o personagem **encolhe e fica cinza**.

Passo 6 — a mensagem de acordo com a resposta

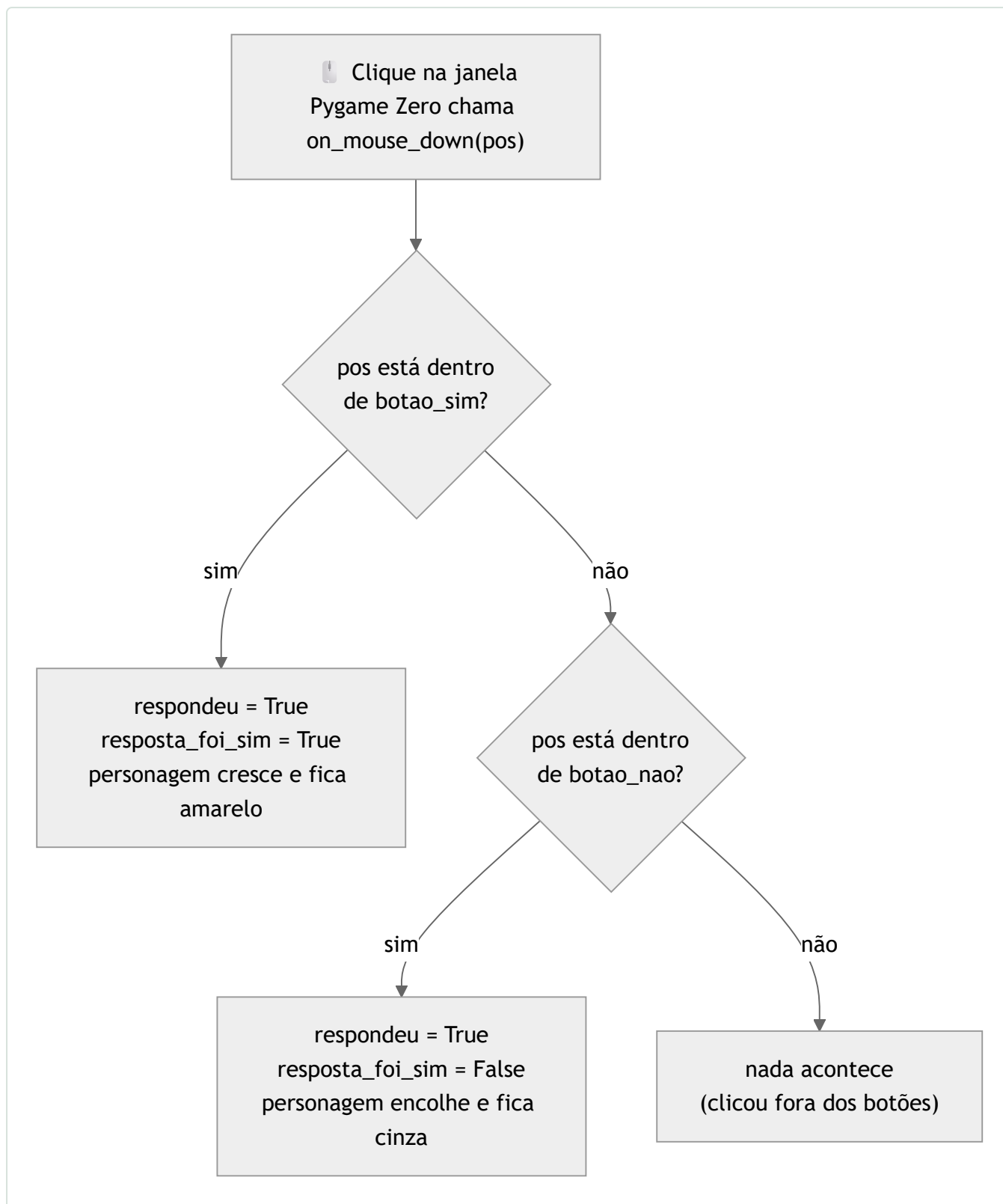
Por último, dentro de `draw()`, depois do personagem e dos botões, mostre uma mensagem só depois que o jogador responder:

```
if respondeu:
    if resposta_foi_sim:
        mensagem = "Muito bem! Continue assim!"
    else:
        mensagem = "Que tal fazer isso hoje?"

    screen.draw.text(
        mensagem,
        center=(WIDTH // 2, HEIGHT // 2 + 220),
        fontsize=26,
        color=COR_DO_TEXTO,
    )
```

- O primeiro `if respondeu:` só entra **depois** que algum botão foi clicado — antes disso, `respondeu` é `False` e nada é desenhado ali.
- O `if / else` de dentro escolhe **qual texto** mostrar, guardando o resultado numa variável `mensagem` antes de desenhar.

Rode. Antes de clicar, só aparecem a pergunta, o personagem e os botões. Clique no **"Sim"**: o personagem cresce, fica amarelo, e aparece **"Muito bem! Continue assim!"** embaixo. Feche, rode de novo e clique no **"Não"**: o personagem encolhe, fica cinza, e aparece **"Que tal fazer isso hoje?"**.



Pronto — compare com o [exemplo.py](#) deste módulo: é o mesmo código que você acabou de construir.

Sua missão

- [] Escolha a cor e o tamanho inicial do seu personagem;
- [] Escolha uma pergunta sobre um hábito cristão (ex.: "Você obedeceu aos seus pais hoje?", "Você leu a Bíblia hoje?");

- [] Programe a reação: **cresce e muda de cor** no "Sim", **fica do mesmo tamanho ou menor** no "Não";
- [] Mostre uma mensagem diferente para cada resposta.

Construa **do zero**, seguindo os passos acima — não copie o `exemplo.py` pronto; ele é só para comparar no fim ou para destravar se você empacar.

Desafios

- **Desafio extra:** adicione uma segunda pergunta que aparece depois da primeira ser respondida.
- **Desafio criativo:** personalize as cores, o formato do personagem e as mensagens.

Palavras novas

Termo	O que significa
Coordenada	Um endereço na tela: um valor <code>x</code> (horizontal) e um valor <code>y</code> (vertical), a partir do canto superior esquerdo
<code>topleft=(x, y)</code>	Outro jeito de posicionar texto: ancora pelo canto superior esquerdo, em vez de centralizar como <code>center</code>
<code>screen.draw.filled_circle(...)</code>	Desenha um círculo preenchido: centro, raio e cor
<code>Rect((x, y), (largura, altura))</code>	Um retângulo: canto superior esquerdo + tamanho
<code>screen.draw.filled_rect(...)</code>	Pinta um <code>Rect</code> inteiro de uma cor
<code>retangulo.center</code>	Atalho do <code>Rect</code> que já calcula o ponto central dele
<code>True</code> / <code>False</code>	Verdadeiro ou falso — os dois únicos valores possíveis de uma variável lógica
<code>if</code> / <code>elif</code> / <code>else</code>	"Se" isso for verdade, faça uma coisa; "senão, se" outra coisa for verdade, faça outra; "senão", faça a última
<code>==</code>	Compara dois valores (é diferente de <code>=</code> , que guarda um valor numa variável)
Evento	Algo que acontece durante o jogo, como um clique na tela
<code>on_mouse_down(pos)</code>	Função especial que o Pygame Zero chama sozinho a cada clique, entregando a posição <code>pos</code> do clique
<code>retangulo.collidepoint(pos)</code>	Responde <code>True</code> / <code>False</code> : esse ponto está dentro do retângulo?
<code>global</code>	Dentro de uma função, avisa que uma variável não é nova — é a de fora, e queremos mudar ela de verdade

Deu erro? Tenta isso primeiro

- **Esqueceu os dois pontos (:) depois do `if` / `elif` / `else`**: confira se colocou `:` no final da linha.

- **O clique não faz nada:** confira se a função se chama **exatamente** `on_mouse_down` (com esse nome, sem erro de digitação) e se recebe `pos` entre os parênteses.
- `UnboundLocalError` **ou a tela não muda depois do clique:** falta o `global` no começo de `on_mouse_down`, listando as variáveis que a função precisa mudar.
- **Usou `=` em vez de `==`:** `=` guarda um valor; `==` compara dois valores — são diferentes!
- **Botão no lugar errado:** confira os números do `Rect` — o primeiro par é a posição, o segundo é o tamanho.

Ao final, **salve** o `jogo.py` (na raiz de `crescendo-como-jesus/`).